

Knome Platform API

Complete Backend Architecture, Technical Specification & API Documentation

TARGET FRAMEWORK

.NET 10.0 (net10.0)

ARCHITECTURE PATTERN

Clean N-Tier Monolith

DATABASE ENGINE

SQL Server (EF Core
10)

AUTHENTICATION

JWT HS256 + BCrypt

1. Executive Architectural Overview

Knome is an enterprise-grade Knowledge Management & Hybrid Social Collaboration Platform engineered for **MPOnline Limited**. The backend application `Backend/Knome.API` is built on Microsoft **ASP.NET Core 10.0**, delivering ultra-fast response times, scalable RESTful APIs, real-time SignalR push notifications, and strict enterprise security controls.

Core Architecture Layers

- **Presentation Layer:** Thin REST Controllers (`Controllers/`) returning standard `ApiResponse<T>` envelopes.
- **Service Layer:** Encapsulated Business Logic (`Services/`) enforcing validation, permissions, and moderation workflows.
- **Data Access Layer:** Generic Repository Pattern (`Repositories/`) over EF Core 10 `KnomeDbContext`.

Key Non-Functional Requirements

- **Security:** JWT HS256 authentication, BCrypt work factor 11 password hashing, role-based authorization (RBAC), and Suspension Guard middleware.
- **Performance:** Compiled EF Core queries, async/await I/O, SignalR Websockets, and background worker expiration.
- **Data Integrity:** Centralized transaction management and FluentValidation pipeline filters.

2. Technology Stack & Component Specs

LAYER / CONCERN	TECHNOLOGY CHOICE	VERSION / IMPLEMENTATION DETAILS
Framework	ASP.NET Core Web API	<code>net10.0</code> (.NET 10.0 SDK)
Database & ORM	SQL Server + Entity Framework Core	<code>Microsoft.EntityFrameworkCore.SqlServer 10.0</code>
Authentication	JWT Bearer + BCrypt.Net	<code>System.IdentityModel.Tokens.Jwt</code> + <code>BCrypt.Net-Next 4.0.3</code>
Real-Time Messaging	ASP.NET Core SignalR	<code>/hubs/notifications</code> Hub Endpoint
Object Mapping	AutoMapper	AutoMapper Profiles across 12 Modules
Input Validation	FluentValidation	Automatic pipeline validation in <code>Validators/</code>
Background Processing	IHostedService Worker	<code>Background/JobExpiryHostedService</code>
File Storage	Local File System Storage	<code>Services/LocalFileStorageService</code> under <code>wwwroot/uploads</code>

3. Application Startup & Pipeline Architecture

The backend initialization workflow resides in `Program.cs`, `Extensions/ServiceCollectionExtensions.cs`, and `Extensions/ApplicationBuilderExtensions.cs`.

3.1 Dependency Injection Assembly (`AddInfrastructure`)

All framework services, authentication schemes, Swagger OpenAPI documentation, and application components are registered via `AddInfrastructure(IServiceCollection, IConfiguration)` :

```
public static IServiceCollection AddInfrastructure(this IServiceCollection services, IConfiguration configur
{
    // 1. Database Wiring (EF Core SQL Server)
    services.AddDbContext<KnomeDbContext>(options =>
        options.UseSqlServer(configuration.GetConnectionString("DefaultConnection")));

    // 2. Authentication & Authorization (JWT HS256)
    services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
        .AddJwtBearer(options => { ... });

    // 3. Application Services & Repositories Registration
    services.AddScoped(typeof(IRepository<>), typeof(Repository<>));
    services.AddScoped<IUserRepository, UserRepository>();
    services.AddScoped<IUserService, UserService>();
    // ... registers all 16 Services & 14 Repositories

    // 4. SignalR & Background Hosted Worker
    services.AddSignalR();
    services.AddHostedService<JobExpiryHostedService>();

    return services;
}
```

3.2 Middleware Execution Pipeline (`UseInfrastructure`)

The request pipeline sequence is strictly ordered in `ApplicationBuilderExtensions.cs` to guarantee security and exception safety:

1. `ExceptionHandlerMiddleware` : Catches all unhandled exceptions globally and formats a unified JSON response envelope.
2. `SecurityHeadersMiddleware` : Injects HTTP security headers (CSP, X-Frame-Options, X-Content-Type-Options, HSTS).
3. `UseCors("AllowFrontend")` : Permits cross-origin requests from the React frontend server.
4. `UseAuthentication()` & `UseAuthorization()` : Evaluates JWT claims and user identity.
5. `SuspensionGuard` : Verifies if the authenticated user account has been suspended by System Admin.
6. `MapControllers()` & `MapHub<NotificationHub>("/hubs/notifications")` : Endpoints execution.

4. Controllers & REST API Endpoints Reference

The backend exposes **16 REST API Controllers** inheriting from `KnomeControllerBase.cs`. Below is the complete listing of all API endpoints, HTTP verbs, and access controls.

4.1 Auth & User Management Controllers

METHOD	ENDPOINT ROUTE	CONTROLLER & HANDLER	ROLES REQUIRED	DESCRIPTION
POST	/api/Auth/login	AuthController.Login	Anonymous	Authenticates credentials with BCrypt, generates JWT HS256 token.
GET	/api/User/me	UserController.GetCurrentUser	Authenticated	Fetches detailed profile, designation, skills, and permissions of logged-in user.
PUT	/api/User/profile	UserController.UpdateProfile	Self	Updates user bio, location, phone number, profile photo, and skills.
GET	/api/User	UserController.GetAllUsers	Authenticated	Lists organization directory with search, department filtering, and pagination.
PUT	/api/User/{id}/role	UserController.UpdateRole	System Administrator	Changes user role assignment (Employee, Community Admin, HR Admin, System Admin).
POST	/api/User/{id}/suspend	UserController.SuspendUser	System Administrator	Suspends an employee account for specified duration with reason log.

4.2 Content & Media Publishing Controllers

METHOD	ENDPOINT ROUTE	CONTROLLER & HANDLER	DESCRIPTION & WORKFLOW
GET	/api/Posts	PostController.GetPosts	Fetches micro-posts feed with author details, media attachments, and reaction counts.
POST	/api/Posts	PostController.CreatePost	Creates a text/media post with image/video attachments and community tagging.
GET	/api/Articles	ArticleController.GetArticles	Returns long-form articles with HTML content, cover image, and calculated reading time.
POST	/api/Articles	ArticleController.CreateArticle	Publishes an article draft with tags, attachments, and version tracking.
GET	/api/Videos	VideoController.GetVideos	Aggregates platform videos (direct uploads, MS Stream/OneDrive embeds).
POST	/api/Videos	VideoController.UploadVideo	Uploads video file. <i>Non-admin uploads trigger Pending Admin Approval workflow.</i>
GET	/api/Podcasts	PodcastController.GetPodcasts	Lists audio podcast episodes, standalone audio files, and series groupings.
POST	/api/Podcasts	PodcastController.CreatePodcast	Publishes audio file or browser recording. <i>Requires Admin approval if non-admin.</i>
POST	/api/Media/upload	MediaController.UploadFile	Stores raw multipart files to disk storage and returns resolved URL.

4.3 Communities, Feed & Interaction Controllers

METHOD	ENDPOINT ROUTE	CONTROLLER & HANDLER	DESCRIPTION
GET	/api/Community	CommunityController.GetCommunities	Lists enterprise communities with member counts, category, and privacy level.
POST	/api/Community	CommunityController.CreateCommunity	Creates a new community hub (Public/Private/Invite-Only).
POST	/api/Community/{id}/join	CommunityController.JoinCommunity	Joins public community or submits join request for private community.
POST	/api/Interaction/react	InteractionController.AddReaction	Adds or toggles reaction (Like, Celebrate, Insightful, Support) on posts/articles.
POST	/api/Interaction/comment	InteractionController.AddComment	Posts a comment or nested reply on posts, articles, or videos.
GET	/api/Feed/hot	FeedController.GetHotPosts	Calculates top trending posts using engagement score & time-decay algorithm.
GET	/api/Karma/balance	KarmaController.GetBalance	Retrieves user total karma points, earned badge tier, and transaction history.
GET	/api/Notifications	NotificationsController.GetNotifications	Fetches current user notifications with unread counts and SignalR sync.

5. Core Business Modules & Logic Architecture

Source-of-Truth Architecture: All business logic is encapsulated inside single-responsibility Service classes located in `Services/`. Repositories in `Repositories/` deal exclusively with EF Core query generation and SQL execution.

5.1 Media Approval & Admin Moderation Workflow

To ensure enterprise compliance and content safety, a multi-tier moderation workflow is enforced across media uploads (Videos and Podcasts):

- Role Check:** When a user calls `/api/Videos` or `/api/Podcasts`, `VideoService` / `PodcastService` evaluates user role.
- Admin Direct Upload:** If the uploader possesses an Admin role (`System Administrator` , `HR Administrator` , `Community Administrator`), the content is immediately saved as `Approved` and published.
- Employee Queueing:** If uploaded by a standard `Employee`, the media is saved with status `PendingApproval`. An automated notification alert is generated for System Administrators.
- Admin Review:** Admins access the **Media Approvals Queue** in `AdminConsole` to preview audio/video metadata and click **Approve** or **Reject**.
- Publication & Notification:** Approving the item publishes it to public feeds and sends a real-time SignalR notification to the author.

5.2 Hot Posts Ranking & Time-Decay Engine (`FeedService.cs`)

The platform computes trending content dynamically using an engagement gravity formula:

```
HotScore = (Reactions * 1.0 + Comments * 2.0 + Shares * 3.0 + Bookmarks * 2.5) / Math.Pow(AgeHours + 2, 1.5)
```

This ensures recent high-engagement posts rank highest on the Dashboard while older posts naturally decay over time.

5.3 Gamification & Karma Points Engine (`KarmaService.cs`)

Employee engagement is incentivized via automated Karma point allocations:

- **Post Creation:** +2 Karma Points (Max 10 pts/day)
- **Article Publication:** +10 Karma Points (Max 30 pts/day)
- **Video Upload:** +8 Karma Points
- **Comment Added:** +2 Karma Points
- **Reaction Received:** +1 Karma Point

6. Database Schema & Scaffolded Entity Models

The backend communicates with SQL Server database `Knome` through 38 scaffolded Entity Framework Core entity classes in `Models/`:

ENTITY MODEL	TABLE NAME	PRIMARY KEY	KEY FOREIGN RELATIONSHIPS
User.cs	Users	UserId (int)	RoleId (Role), DepartmentId (Department)
Post.cs	Posts	PostId (int)	AuthorUserId (User), CommunityId (Community)
Article.cs	Articles	ArticleId (int)	AuthorUserId (User), CategoryId (Category)
Video.cs	Videos	VideoId (int)	UploaderUserId (User), CategoryId (Category)
Podcast.cs	Podcasts	PodcastId (int)	UploaderUserId (User), SeriesId (PodcastSeries)
Community.cs	Communities	CommunityId (int)	CreatorUserId (User), CategoryId (Category)
KarmaBalance.cs	KarmaBalances	UserId (int)	UserId (User)
AuditLog.cs	AuditLogs	AuditId (long)	UserId (User)

7. Build Verification & Development Commands

The backend project builds with **0 Errors and 0 Warnings** targeting .NET 10.0:

```
# Run API Server locally on http://localhost:5095/swagger
cd Backend/Knome.API
dotnet build -nologo
dotnet run

# Run Full DI Resolution Check across all 14 controllers & 12 modules
dotnet run --project "Tools/VerifyDiResolvers/VerifyDiResolvers.csproj"
```